UNIVERSIDAD CATÓLICA BOLIVIANA «SAN PABLO»

Carrera de Ingeniería en Inteligencia Artificial

EQUIPEGO

Simulación concurrente de despacho y entregas en la Zona Equipetrol

INFORME FINAL DE PROYECTO

Computación Concurrente, Paralela y Distribuida • C++17

Proyecto de simulación con OpenCV, JSON y biblioteca estándar de C++

Autor: Leandro Emiliano Miranda Román

Sede: Santa Cruz de la Sierra, Bolivia

Fecha: 20 de octubre de 2026

Repositorio: github.com/Leandro0210/EquipeGo

Resumen ejecutivo

EquipeGo es un simulador de reparto urbano concurrente desarrollado en C++17. Representa una flota de motocicletas que recibe pedidos generados de forma aleatoria, calcula el repartidor más conveniente mediante estimaciones de tiempo de entrega (ETA), atiende restaurantes con cupos limitados, recorre un grafo de calles y registra incidencias, entregas, rechazos y pedidos pendientes. Su interfaz utiliza OpenCV sobre una captura recortada de OpenStreetMap correspondiente a Equipetrol, Santa Cruz de la Sierra [R1, R2].

La arquitectura diferencia responsabilidades entre productor, libro único de pedidos, despachador, pool de cotizaciones, flota de repartidores, coordinador de rescate, monitores de restaurante, bitácora de eventos e interfaz gráfica. Las tareas avanzan de manera concurrente; los ETA de distintos candidatos se distribuyen entre workers para admitir ejecución paralela, sujetos al hardware disponible.

El escenario predeterminado consta de 30 nodos, 42 segmentos viales, 7 tramos unidireccionales, 6 restaurantes y 8 repartidores; la flota inicial posee una bolsa de 3 pedidos por moto. La simulación está parametrizada en un archivo JSON y admite otros escenarios válidos [R2].

Evidencia	Resultado documentado
Validación de tres compilaciones	CTest Release 10/10; TSAN 10/10; ASAN+UBSAN 10/10 (30 comprobaciones acumuladas).
Instrumentación final	12/12 ejecuciones; 5.878 pedidos creados; 96 averías; cierre máximo observado 0,265 s.
Casos adversarios	22/22: 12 configuraciones inválidas, 7 escenarios de carga, 3 apagados por señales.
Interfaz con 64 motos	600 s simulados; 120 creados, 59 entregados, 61 pendientes, 0 rechazados en esa corrida.
Optimización ETA con 64 motos	CPU media 1,99 s antes versus 1,61 s después (−19,16 %); tres repeticiones por caso.

Las cifras son resultados registrados en el repositorio; no se han vuelto a ejecutar las pruebas al redactar este documento. Se diferencia explícitamente la validación previa a la entrega de las tareas de verificación todavía pendientes.

Índice de contenidos

1. Introducción y planteamiento del problema
2. Objetivos, alcance y metodología
3. Requisitos funcionales y restricciones
4. Arquitectura y conexiones entre componentes
5. Descripción técnica por módulos y archivos
6. Diseño concurrente, paralelismo y sincronización
7. Algoritmos, eventos e invariantes
8. Interfaz visual y mapa OpenStreetMap
9. Compilación, ejecución y cierre ordenado
10. Pruebas, experimentos y resultados
11. Limitaciones y aspectos pendientes
12. Preparación para la defensa y modificaciones
13. Conclusiones
Referencias y anexos

1. Introducción y planteamiento del problema

Un servicio de entregas debe atender solicitudes mientras sus motociclistas recorren rutas, los restaurantes preparan pedidos y el sistema decide nuevas asignaciones. Las actividades transcurren en tiempos que se solapan; además, comparten recursos escasos, como los cupos de recogida, las bolsas de los repartidores y la cola central de pedidos. Estas características convierten la simulación en un caso pertinente para aplicar concurrencia, paralelismo y mecanismos de sincronización [R1].

Una solución secuencial obligaría a esperar cada desplazamiento o preparación antes de procesar el resto. EquipeGo adopta una descomposición en hilos especializados, mientras que los datos compartidos se protegen con bloqueos de ámbito, variables de condición, tipos atómicos y futuros. El resultado busca preservar la consistencia sin utilizar bucles de espera activa para coordinar hilos.

1.1. Problema técnico

Mantener un flujo continuo y coherente de pedidos bajo cargas variables y averías, sin exceder capacidades, duplicar entregas ni bloquear el apagado. Se añade la exigencia de que las motos se visualicen en segmentos reales del grafo y de que la secuencia de eventos pueda auditarse al finalizar.

1.2. Fundamentación

El proyecto sigue el trabajo final “Equipetrol Delivery: una simulación de despacho concurrente” del repositorio docente y utiliza como referencias formativas las carpetas 01 a 07 del curso LIA-221 [R1, R3]. La solución está implementada con C++17, OpenCV y nlohmann/json; CMake organiza la compilación [R2].

2. Objetivos, alcance y metodología

2.1. Objetivo general

Desarrollar y validar una simulación concurrente de despacho de pedidos en Equipetrol que coordine repartidores independientes, estimaciones ETA paralelizables, recursos limitados, incidentes y visualización en tiempo real, manteniendo consistencia observable en el registro de eventos.

2.2. Objetivos específicos

- Modelar el mapa como un grafo dirigido de intersecciones y calles, respetando tramos de un solo sentido y configuraciones JSON alternativas.
- Producir pedidos reproducibles para una semilla, con intervalos aleatorios y ráfagas, y gestionar su ciclo de vida sin duplicarlos.
- Ejecutar cada repartidor en su propio hilo, coordinar la selección ETA mediante un pool y limitar la ocupación de restaurantes y bolsas.
- Resolver averías mediante transferencia completa de pedidos y coordinar el cierre ante tiempo, cierre gráfico o señales.
- Registrar JSONL y resumen final consistentes; evaluar corrección, rendimiento y posibles carreras mediante pruebas y sanitizadores.

2.3. Alcance

La simulación trabaja sobre una configuración y una imagen fija de Equipetrol; no consulta información de tráfico en tiempo real, no administra pagos y no es un servicio de producción conectado a comercios reales. Representa tiempos acelerados, rutas y eventos simulados. Incluye herramientas de prueba y un ejecutable auxiliar para escenarios instrumentados.

2.4. Método de trabajo

El desarrollo se organizó por avances funcionales DP.0 a DP.12, con integración incremental, escenarios de prueba, instrumentación y documentación. Se analizaron fuentes del propio repositorio y registros de pruebas; los valores que aparecen en las tablas se atribuyen a la evidencia conservada [R2, R4, R5].

3. Requisitos funcionales y restricciones

Área	Comportamiento implementado	Fuente principal
Ciudad y configuración	Imagen OSM; nodos, calles, restaurantes, límites geográficos; validación JSON.	ConfigLoader.cpp; RoadNetwork.cpp; equipetrol.json
Pedidos	Intervalos exponenciales; ráfagas de 1 a burstMax; cola acotada; estados y rechazos.	OrderGenerator.cpp; OrderProducer.cpp; OrderBook.cpp; Order.hpp
Flota	N hilos Courier; límite bagCapacity; espera bloqueante; viajes y entregas.	Fleet.cpp; Courier.cpp
Restaurantes	pickupSlots, tickets FIFO y permisos RAII.	RestaurantManager.hpp; PickupGate.hpp
Despacho	Cotizaciones ETA; selección por mínimo; quoteTimeoutMs; acceptTimeoutMs.	Dispatcher.cpp; DispatchLoop.cpp
Incidentes	Avería pseudoaleatoria por recorrido; transferencia completa bajo dos mutex.	BreakdownPolicy.hpp; Courier.cpp; Fleet.cpp
Observabilidad	Eventos JSONL, estadísticas y auditoría de balance final.	Logger.cpp; main.cpp
Interfaz y apagado	OpenCV en hilo principal; snapshots; SIGINT/SIGTERM y join.	SimulationDashboard.cpp; main.cpp

3.1. Restricciones tecnológicas

La rúbrica exige C++17, dependencias de visualización y JSON, y concurrencia implementada con biblioteca estándar, sin OpenMP, TBB o paquetes externos de planificación de hilos. `CMakeLists.txt` fija C++17, utiliza Threads, OpenCV ≥ 4.1 y nlohmann/json, y aplica `-Wall -Wextra`. La opción ENABLE_TSAN habilita instrumentación para ThreadSanitizer [R1, R2].

4. Arquitectura y conexiones entre componentes

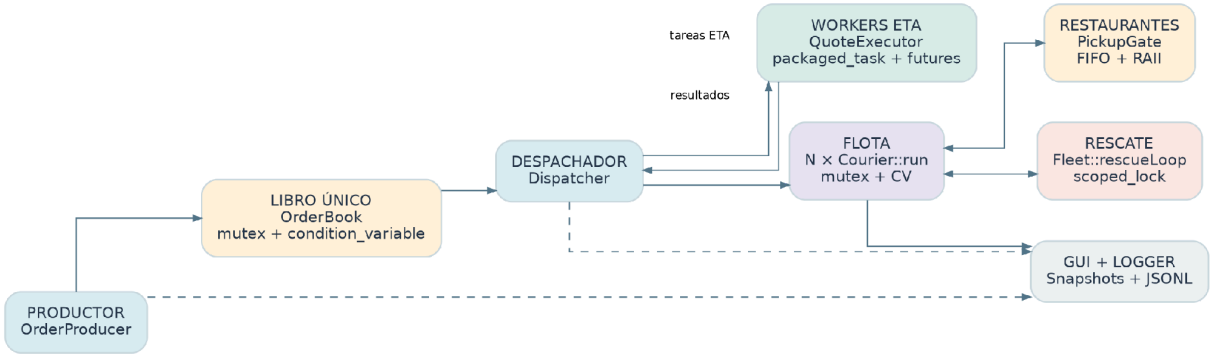


Figura 1. Arquitectura lógica, canales de intercambio y monitores. Elaboración propia basada en los archivos de la rama master [R2].

La simulación tiene un hilo principal dedicado a la GUI, un productor, un despachador, un coordinador de rescate, N hilos de repartidores y W trabajadores ETA. Las dependencias se inyectan desde `main()` mediante referencias o punteros gestionados, por lo que un mismo `OrderBook` y una misma flota son compartidos bajo reglas de sincronización.

4.1. Hilos de ejecución y responsabilidades

Hilo / cantidad	Trabajo durante la simulación	Punto de creación / cierre
Principal (1)	Dibuja GUI, lee snapshots, controla duración y señales.	main.cpp; fin con cierre y resumen.
Productor (1)	Espera intervalo; crea ráfagas; inserta o rechaza.	OrderProducer::run; main.cpp std::thread; join.
Despachador (1)	Extrae pedidos; cotiza; selecciona; registra asignaciones.	DispatchLoop.cpp::run; main.cpp; join.
Repartidores (N)	Espera; recorre calles; recoge; entrega; reporta averías.	Fleet.cpp::start → Courier::run; Fleet::join.
Rescate (1)	Busca bolsas de motos averiadas y receptoras disponibles.	Fleet.cpp::startRescue → rescueLoop; join.

Hilo / cantidad	Trabajo durante la simulación	Punto de creación / cierre
ETA (W)	Ejecuta tareas de cálculo con resultados future.	Dispatcher.cpp::QuoteExecutor; destructor y join.

W se calcula como max(1, min(N, min(max(2, hardware_concurrency()), 8))). Con 8 motos y hardware suficiente es 8; con 64 motos el máximo sigue siendo 8. Las tareas del resto se encolan; no se promete ejecución simultánea literal de todos los candidatos.

4.2. Secuencia de un pedido

1. OrderGenerator calcula el intervalo aleatorio y el tamaño de la ráfaga; OrderProducer asigna ID, restaurante, nodo de destino y tiempo.
2. RoadNetwork determina si la entrega es alcanzable; si no, se registra unreachable.
3. OrderBook protege la cola bajo bookMutex; si supera el máximo, el productor registra queueFull.
4. DispatchLoop extrae pedidos y pide ETA a QuoteExecutor, que distribuye los cálculos entre W workers.
5. Se ordenan las respuestas válidas por ETA y se intenta reservar capacidad en Courier::tryAssignOrder.
6. Courier::run espera la preparación, adquiere Permit FIFO en PickupGate, recoge la comida, viaja por el grafo y entrega.
7. EventLogger serializa cada evento; main detiene los hilos, audita el registro y produce el balance final.

5. Descripción técnica por módulos y archivos

5.1. Configuración y ciudad

Archivo	Responsabilidad concreta
src/config/Config.hpp	Define SimulationConfig, MapBounds, Node, Street, Restaurant y los parámetros por subsistema.
src/config/ConfigLoader.cpp	Abre JSON, valida campos y referencias, carga la imagen, transforma lat/lon en píxeles y calcula longitudes.
src/map/RoadNetwork.hpp / .cpp	Construye adyacencias del grafo; isReachable delega en shortestRoute, basado en Dijkstra.
config/equipetrol.json	Escenario predeterminado, semilla, tiempos, tamaño de flota, capacidades, restaurantes, nodos y calles.

5.2. Pedidos y productor

Archivo	Responsabilidad concreta
orders/Order.hpp	Define estados de pedido y transiciones legales; copia eliminada y movimiento permitido.
orders/OrderGenerator.hpp / .cpp	Generación reproducible mediante mt19937; intervalos exponenciales y ráfagas uniformes.
orders/OrderProducer.hpp / .cpp	Hilo generador, comprobación de alcanzabilidad, inserción en OrderBook y rechazos.
orders/OrderBook.hpp / .cpp	Monitor con deque<Order>, bookMutex, ordersCv, inFlight, límite maxPending y cierre.

5.3. Despacho, flota y restaurantes

Archivo	Responsabilidad concreta
dispatch/Dispatcher.hpp / .cpp	QuoteExecutor; workers ETA; futuros, tiempos límite, excepciones; caché compartida.
dispatch/DispatchLoop.cpp	Consumidor OrderBook; cola local waiting; ordena ETA, asigna y vence por noCourier.
dispatch/SharedDistanceCache.hpp	Memoización de distancias restaurante→cliente con promise/shared_future.
fleet/Fleet.hpp / .cpp	Propiedad de Courier, hilos de flota, rescate, CapacitySignal y RestaurantManager.
fleet/Courier.hpp / .cpp	Estado, bolsa, movimiento, preparación, entrega, avería y transferencia de pedidos.

Archivo	Responsabilidad concreta
fleet/CapacitySignal.hpp	Notificaciones de cambio basadas en generación, mutex y condition_variable.
incidents/BreakdownPolicy.hpp	Plan de avería pseudoaleatorio, derivado de seed+repartidor+pedido+etapa.
restaurants/PickupGate.hpp	Monitor de cupos con FIFO de tickets y Permit de liberación automática.
restaurants/RestaurantManager.hpp	Instancia un PickupGate por restaurante y comparte referencias con la flota.
restaurants/PreparationTime.hpp	Tiempo de preparación determinista en el intervalo configurado por restaurante.

5.4. Observabilidad, visualización y pruebas

Archivo	Responsabilidad concreta
core/Clock.hpp	Traduce tiempo real estable a milisegundos simulados mediante timeScale.
core/Logger.hpp / .cpp	Log JSONL y métricas protegidos por logMutex; snapshot de contadores.
visualization/MapViewer.hpp / .cpp	Dibujo de mapa base, calles, intersecciones y restaurantes.
visualization/SimulationDashboard.hpp / .cpp	GUI principal y muestreo de snapshots de repartidores; animación por interpolación.
visualization/*_preview.cpp	Programas auxiliares de vista previa y pruebas de integración.
src/main.cpp	Composición, señales de cierre, inicio de hilos, auditoría y resumen final.
tests/ y benchmarks/	Pruebas automatizadas, campañas dinámicas y experimentos reproducibles.
evidencias/	Conserva configuraciones, reportes, logs, resúmenes, capturas y registros crudos.

6. Concurrencia, paralelismo y sincronización

6.1. Independencia de repartidores: std::thread

```
for (const auto &courier : couriers) {
    threads.emplace_back(&Courier::run, courier.get());
}
```

Fleet::start() crea un hilo por moto; la instancia Courier se mantiene viva mediante unique_ptr en Fleet. El método Courier::run() conserva el ciclo individual de trabajo. El despachador, el productor, el rescate y la GUI siguen progresando mientras las motos viajan [R2].

6.2. El libro único: monitor productor-consumidor

```
std::lock_guard<std::mutex> lock(bookMutex);
if (stopRequested || orders.size() + inFlight >= maxPending)
    return false;
orders.push_back(std::move(order));
```

La exclusión mutua une comprobación de capacidad e inserción, evitando carreras sobre la cola. Para extraer pedidos se usa ordersCv.wait(lock, predicado), que duerme al consumidor y libera el mutex mientras espera; notify_one lo despierta cuando llega trabajo. El contador inFlight incorpora pedidos extraídos todavía no resueltos.

6.3. Espera sin polling en Courier

```
std::unique_lock<std::mutex> lock(stateMutex);
workCv.wait(lock, [this] {
    return stopRequested || state == CourierState::Broken ||
           destination.has_value() || !assignedOrders.empty();
});
```

Cada hilo de repartidor queda bloqueado sin ejecutar consultas continuas. El mismo stateMutex protege estado, bolsa y posición. Un notify_one posterior a una asignación o notify_all durante el cierre despierta el hilo; el predicado tolera despertares espurios.

6.4. Paralelismo en QuoteExecutor

```
std::unique_lock<std::mutex> lock(queueMutex);
queueCv.wait(lock, [this] { return stopping || !jobs.empty(); });
job = std::move(jobs.front());
jobs.pop_front();
lock.unlock();
job();  // El cálculo ETA ocurre fuera del mutex.
```

El fragmento expresa el flujo equivalente a workerLoop(): extraer bajo bloqueo y ejecutar fuera. El código fuente real libera el lock al salir de un bloque de ámbito, sin llamar explícitamente a unlock en ese punto. Los ETA de candidatos se encolan como packaged_task<TimedQuote()> y se entregan a workers con future; el despachador espera sobre un deadline común mediante future.wait_until [R2].

6.5. Canal de cálculo compartido: promise / shared_future

SharedDistanceCache guarda un shared_future por par dirigido (restaurante, cliente). Un único worker publica el cálculo para ese par; otros esperan el mismo resultado o su excepción. La caché bloquea brevemente el mapa de futuros y ejecuta Dijkstra fuera del mutex. No se usa std::async(std::launch::async) literalmente; la política de lanzamiento es un pool fijo.

6.6. FIFO en restaurantes: mutex, CV y RAII

```
cv_.wait(lock, [this, myTicket] {
    return stopping_ ||
          (myTicket == nextToEnter_ && occupied_ < capacity_);
});
++nextToEnter_;
++occupied_;
return Permit(this);
```

El ticket fija el orden de admisión al monitor y occupied_ nunca se incrementa sin exclusión mutua. Cada restaurante cuenta con un PickupGate distinto, pero todas las motos que llegan a ese restaurante acceden al mismo objeto. Permit no se copia y su destructor devuelve el cupo, incluso ante retornos anticipados.

6.7. Rescate atómico con dos mutex

```
std::scoped_lock lock(stateMutex, recipient.stateMutex);
if (recipientReserved > capacity || outgoing > capacity - recipientReserved)
    return false;
// Se comprueban IDs y se prepara staged.
recipient.assignedOrders.swap(staged);
activeOrder.reset();
assignedOrders.clear();
blockedOrders.clear();
```

El rescate necesita modificar las bolsas de dos Couriers. scoped_lock adquiere ambos mutex con una estrategia que evita el deadlock por orden opuesto. Se verifica que la bolsa completa quepa antes de moverla, y se publica la transferencia mientras se mantienen ambos bloqueos. Desde la perspectiva de lectores sincronizados, no existe un estado intermedio visible. El coordinador de rescate es un hilo único; ante falta de capacidad espera un cambio sin sondeo activo.

6.8. Inventario de recursos compartidos

Recurso	Protección	Riesgo evitado
OrderBook: deque / inFlight	bookMutex + ordersCv	Corrupción, pérdida de trabajo y sobrecapacidad
Courier: bolsa, estado, posición	stateMutex + workCv	Asignación doble, inconsistencias visuales
Rescate entre dos Couriers	std::scoped_lock(dos mutex)	Transferencia parcial observable y deadlock
PickupGate: plazas y tickets	mutex_ + cv_ + Permit	Sobrecupo, orden FIFO vulnerado
QuoteExecutor: cola ETA	queueMutex + queueCv	Extracción simultánea no coordinada
SharedDistanceCache: mapa	mutex_ + shared_future	Repetición de Dijkstra / carrera en caché
CapacitySignal: generación	signalMutex + signalCv	Espera activa y pérdida de eventos de cambio

Recurso	Protección	Riesgo evitado
EventLogger: archivo y métricas	logMutex	JSONL mezclado y contadores inconsistentes
Indicadores de parada / rescate	atomic<bool> / atomic<size_t>	Lecturas-escrituras simultáneas sin coordinación

7. Algoritmos, eventos e invariantes

7.1. Red vial y rutas mínimas

RoadNetwork crea un grafo de adyacencia en el que las calles no unidireccionales introducen aristas en ambos sentidos, mientras que oneWay conserva solo el sentido declarado. shortestRoute emplea Dijkstra con priority_queue; la posibilidad de alcanzar un destino se expresa con optional<Route>. La distancia se calcula sobre tramos de calle y el ETA divide distancia por velocidad en metros por segundo.

La posición visual no se reconstruye por un temporizador artificial independiente. Courier::travelTo publica un MotionInterval por segmento y getSnapshot calcula la fracción de progreso; SimulationDashboard interpola entre píxeles de extremos. Ante avería congela la fracción alcanzada.

7.2. Generación de demanda y máquina de estados

OrderGenerator usa mt19937 con orders.seed, distribución exponencial para intervalos y uniforme discreta para ráfagas de 1 a burstMax. OrderProducer asigna identificadores consecutivos, marca un tiempo común a cada ráfaga, verifica la alcanzabilidad y crea registros orderCreated u orderRejected. Si el libro se encuentra lleno, el rechazo se denomina queueFull.

```
Created -> Assigned -> PickedUp -> Delivered
     |          |          |
     +-> Rejected           +-> Pending
     +-> Pending (también posible desde Assigned)
```

Order::transitionTo impide volver de un estado final a uno anterior. Los rechazos inmediatos por queueFull y unreachable se registran como eventos sin crear necesariamente un objeto Order con estado Rejected; la auditoría los contabiliza a partir de JSONL.

7.3. Tiempos límite de despacho

quoteTimeoutMs es una espera en milisegundos reales para las cotizaciones; se calcula un único deadline por pedido y future.wait_until lo respeta. Las cotizaciones fuera del plazo se ignoran, aunque el worker puede completar la tarea en segundo plano. acceptTimeoutMs es el plazo simulado para que alguna moto acepte el pedido; si expira, DispatchLoop escribe orderRejected con razón noCourier. Son límites conceptualmente diferentes.

7.4. Manejo de excepciones aisladas

El worker encapsula error de cálculo en std::exception_ptr (std::current_exception). En el despachador, std::rethrow_exception vuelve a lanzar el error dentro de un try/catch, lo registra como QuoteFailure y permite continuar con otras cotizaciones. La capacidad de la flota se comprueba de nuevo bajo mutex antes de asignar definitivamente.

7.5. Invariantes de consistencia y auditoría

```
created = delivered + rejected + pending
sum(deliveredPerCourier) = delivered
cada orderId: a lo sumo una entrega final
OrderBook: orders.size() + inFlight <= maxPending
Courier: reservedCount <= bagCapacity
PickupGate: occupied_ <= pickupSlots
```

main.cpp detiene productor, despachador y flota, recolecta sus pedidos finales, convierte los pendientes y audita el JSONL antes de emitir un JSON resumen. También comprueba duplicados entre los resultados de

Fleet y una única línea final simulationStopping. Estas propiedades están sujetas al alcance de las pruebas ejecutadas; no constituyen una demostración universal de corrección.

7.6. Contrato de eventos

Evento JSONL	Ocasión de registro
orderCreated	El productor genera un identificador de pedido.
orderAssigned	Una moto acepta la asignación del despachador.
pickupStarted	Comienza el uso de un cupo de recogida.
orderPickedUp	Termina la recogida y cambia a PickedUp.
orderDelivered	La moto entrega al cliente.
orderRejected	queueFull, unreachable o noCourier.
courierBreakdown	Se marca una avería durante un trayecto.
orderReassigned	El coordinador mueve un pedido entre motos.
simulationStopping	Último evento al comenzar el cierre final.

8. Interfaz visual y mapa OpenStreetMap

8.1. Procedencia cartográfica y sistema de coordenadas

El autor obtuvo el fondo de la página oficial de OpenStreetMap mediante captura de pantalla y recorte manual; no se conserva el enlace de vista ni la fecha exacta. La atribución “© OpenStreetMap contributors” figura en el JSON, el README y la interfaz. La transformación latitud/longitud a píxeles usa el bounding box declarado para mapear cada nodo a coordenadas de imagen [R2, R6].

Norte	Sur	Oeste	Este
-17.75147	-17.77655	-63.21443	-63.18718

Estos límites son valores declarados en la configuración, no una calibración geográfica medida independientemente. Queda pendiente comprobar que coincidan exactamente con la captura recortada y que los anillos y la avenida San Martín estén representados de manera reconocible.

8.2. Diseño visual del simulador

MapViewer construye el fondo, las vías, los nodos y los restaurantes. SimulationDashboard obtiene copias seguras de CourierSnapshot y EventLogger::snapshotStats, interpola las motos sobre los segmentos del grafo y muestra sus estados mediante colores: gris libre, naranja al dirigirse al restaurante, verde en fase de entrega y rojo si está averiada. Rombos rojos identifican destinos de pedidos activos.

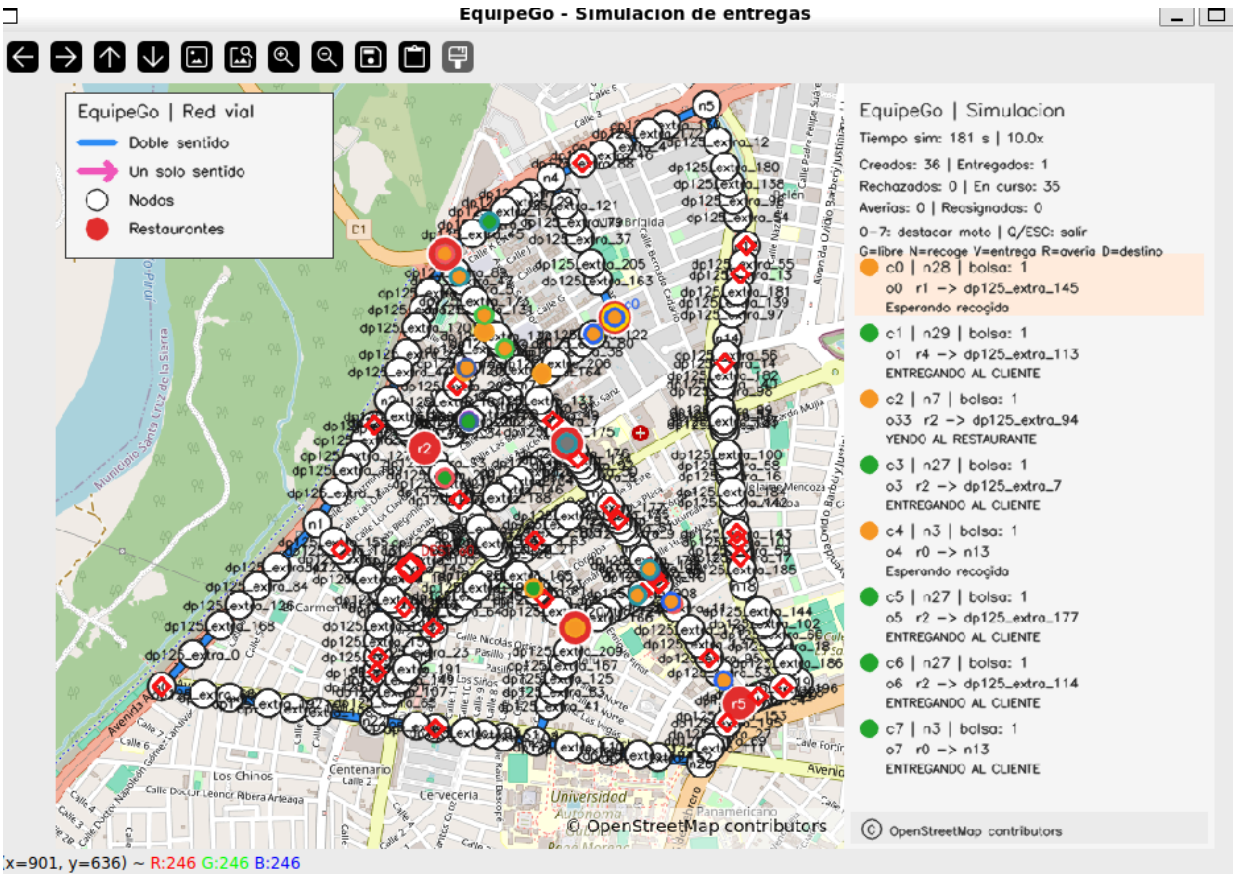


Figura 2. Captura de validación con 64 repartidores (fuente: evidencias/dp12/validacion_visual_64_motos_20261013.png). Se aprecia densidad alta de etiquetas; una captura estática no prueba fluidez temporal.

8.3. Separación entre cálculo y dibujo

La ventana de HighGUI se abre y actualiza únicamente desde el hilo principal, mientras la flota modifica su estado bajo mutex. La GUI no dibuja accediendo directamente a los contenedores mutables: utiliza snapshots copiados. Esto disminuye la contención y evita lecturas concurrentes desprotegidas durante la representación.

9. Compilación, ejecución y cierre ordenado

9.1. Dependencias y comando oficial

Entorno documentado: Ubuntu/WSL2, GCC 15.2.0 y OpenCV 4.10.0 en los ensayos finales. Requisitos declarados: CMake ≥ 3.11, compilador C++17, biblioteca Threads, OpenCV ≥ 4.1 y nlohmann/json ≥ 3.2 (o FetchContent). La interfaz gráfica requiere WSLg/X11 o equivalente [R2].

```
# Desde la raíz del repositorio
mkdir build && cd build
cmake ..
make
./delivery_sim ../config/equipetrol.json

# Registro en ruta alternativa
./delivery_sim ../config/equipetrol.json --log ensayo.jsonl
```

El ejecutable oficial siempre intenta abrir la ventana OpenCV y no admite --headless, --gui, --duration ni --scale. Se pueden utilizar esas banderas en el ejecutable auxiliar de diagnóstico cuando se compile explícitamente mediante -DEQUIPEGO_BUILD_BENCHMARK=ON.

9.2. Prueba rápida y configuraciones alternativas

```
cmake -S . -B build-bench -DEQUIPEGO_BUILD_BENCHMARK=ON
cmake --build build-bench -j4
./build-bench/delivery_sim_bench ./config/equipetrol.json \
```

```
    --headless --scale 10 --duration 300 --log /tmp/prueba.jsonl
```

Para aumentar o reducir número de motos, tasas de pedidos, límites de capacidad, probabilidad de avería o duración, se modifica una copia del archivo JSON, conservando el formato. Nunca deben editarse los registros originales de una prueba ya documentada.

9.3. Apagado sin hilos abandonados

SignalStopController instala SIGINT/SIGTERM y usa un indicador atomic<bool> y, bajo POSIX, un self-pipe para despertar al principal. Después del cierre de ventana, tiempo o señal, main detiene primero al productor, luego al despachador y finalmente a la flota; realiza join de todos, finaliza pendientes y escribe simulationStopping. Así se evita destruir componentes que todavía están siendo usados por trabajadores.

```
producer.requestStop();
producerThread.join();
dispatcher.requestStop(orderBook);
dispatcherThread.join();
fleet.requestStop();
fleet.join();
// Recuperar resultados, auditar JSONL y emitir resumen.
```

10. Pruebas, experimentos y resultados

10.1. Alcance de las pruebas documentadas

Las evidencias están conservadas en el repositorio. Es necesario separar las corridas históricas de DP.11 de la auditoría final del código 0b51082; ambas tienen conjuntos de datos y tiempos de cierre distintos. Las cifras siguientes proceden de los archivos README y evidencias de DP.12, sin afirmar que se ejecutaron de nuevo para este informe [R2, R4, R5].

Verificación final	Resultado registrado	Localización
CTest Release	10/10	evidencias/dp12/auditoria_0b51082/ctest/release.txt
CTest TSAN	10/10	evidencias/dp12/auditoria_0b51082/ctest/tsan.txt
CTest ASAN+UBSAN	10/10	evidencias/dp12/auditoria_0b51082/ctest/asan.txt
Sanitizadores: ejecuciones	12/12; 5.878 pedidos; 96 averías; cierre máx. 0,265 s	evidencias/dp12/auditoria_0b51082/{tsan,asan}/
Adversarias	22/22: 12 invalidaciones, 7 estrés, 3 señales	evidencias/dp12/adversarial/
Contrato público/GUI	Validado; GUI obligatoria y benchmark opcional	tests/test_dp12_8_contract.py
Escenario visual 64 motos	120 creados; 59 entregados; 61 pendientes; 0 rechazados	evidencias/dp12/validacion_visual_64_motos_20261013.png

La campaña histórica DP.11 también registra 12/12 corridas satisfactorias y 5.430 pedidos, 96 averías y un cierre máximo de 0,518 s. Es un antecedente de otra etapa y no debe sumarse ni confundirse con los 5.878 pedidos de la auditoría final [R4].

10.2. Experimento de optimización del ETA (DP.10)

Se comparó el cálculo de rutas repetidas contra una versión con SharedDistanceCache. Condiciones declaradas: tamaños de flota 8, 32 y 64; 900 segundos simulados; timeScale = 60; tres repeticiones por variante; mismo generador, semilla y escenario, cambiando la estrategia para calcular el tramo restaurante→cliente [R2, R5].

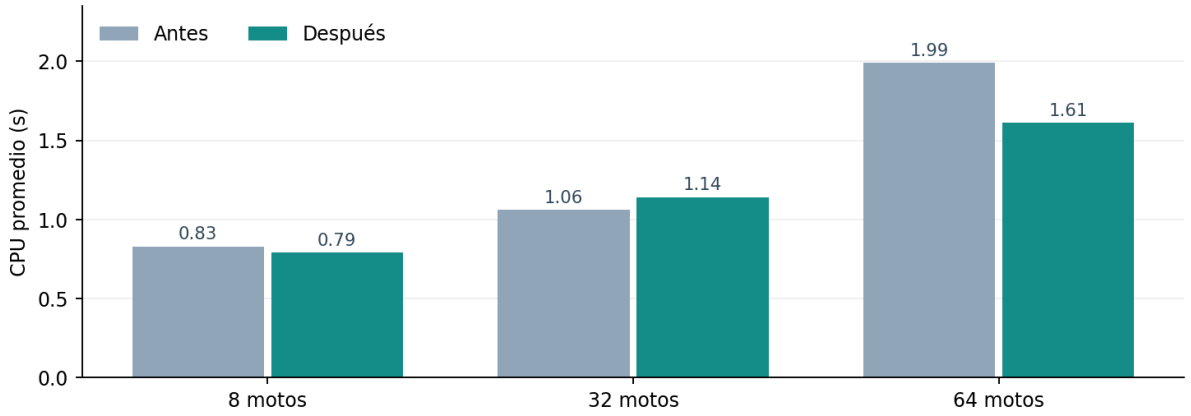


Figura 3. Tiempo de CPU promedio en segundos, antes y después de memoización concurrente del tramo compartido. Datos de evidencias DP.10.

Motos	CPU antes	CPU después	Variación	Entregas promedio antes → después
8	0,83 s	0,79 s	−5,19 %	17 → 17
32	1,06 s	1,14 s	+7,36 %	58 → 58
64	1,99 s	1,61 s	−19,16 %	93,3 → 96,7

El mejor resultado de CPU apareció con 64 motos, con disminución aproximada del 19,16 %. No obstante, con 32 motos la CPU aumentó 7,36 %, por lo que la optimización no mejora todos los escenarios. Las diferencias en entregas reflejan variaciones de planificación y demanda; el experimento no permite concluir mejoras universales de latencia.

10.3. Sanitizadores y pruebas de carrera

Las herramientas empleadas fueron ThreadSanitizer (carreras), AddressSanitizer y LeakSanitizer (memoria y fugas), y UndefinedBehaviorSanitizer (comportamiento indefinido). La auditoría final consignó 6 ejecuciones TSAN y 6 ASAN/UBSAN con registros crudos y sin hallazgos reportados para esos escenarios. La prueba de SharedDistanceCache demuestra además que 32 hilos solicitando el mismo par reutilizan un solo cálculo y que una excepción se comparte con los interesados.

```
# Ejemplo de compilación instrumentada, según README
cmake -S . -B build-tsan -DENABLE_TSAN=ON \
  -DEQUIPEGO_BUILD_BENCHMARK=ON
cmake --build build-tsan -j4
ctest --test-dir build-tsan --output-on-failure
```

La ausencia de reportes no garantiza por sí misma ausencia absoluta de carreras, fugas o bloqueos bajo cualquier intercalado; solo respalda el conjunto de entradas y herramientas probadas.

10.4. Estrés y consistencia del resultado

Se registran 22 escenarios adversarios: 12 invalidaciones de configuración, 7 cargas válidas y 3 pruebas de terminación por señales. La corrida visual de 64 motos, con 600 s simulados y escala 10, produjo 120 pedidos, de los cuales 59 finalizaron entregados y 61 quedaron pendientes, sin rechazos. Es un ejemplo de funcionamiento bajo alta concurrencia y no una garantía sobre todas las cargas posibles.

11. Conclusiones

EquipeGo integra en una sola aplicación los contenidos fundamentales de programación concurrente: hilos explícitos, referencias y movimiento seguro, monitores, variables de condición, futuros y promesas, exclusión mutua de uno y dos recursos, RAII, tipos atómicos y cierre ordenado. La división de responsabilidades permite estudiar qué componente produce, consume, decide, entrega, observa y registra cada evento.

El uso de un pool ETA evita lanzar un hilo nuevo por cada cotización, mientras que SharedDistanceCache elimina cálculos redundantes de rutas compartidas. Las pruebas documentadas muestran consistencia en los

escenarios evaluados y una reducción de CPU para la flota de 64 motos; el comportamiento con 32 motos evidencia que el efecto de optimización depende de la carga.

La principal fortaleza del trabajo para la defensa es la trazabilidad: cada requisito se vincula con clases específicas, protocolos de sincronización y evidencias verificables. Se recomienda cerrar la calibración geográfica del recorte y consultar cualquier interpretación adicional que el docente establezca sobre la política de lanzamiento de tareas. Frente a una modificación de último momento, el enfoque correcto es preservar las invariantes y repetir pruebas focalizadas.

Referencias

[R1] Docente del curso. Equipetrol Delivery: trabajo final y rúbrica en español. Archivo Equipetrol_Delivery_Project_Brief_ES.pdf. Fuente del enunciado y criterios de evaluación.

[R2] Miranda Román, L. E. EquipeGo, rama master, commit documental 668f346; código funcional auditado 0b51082. https://github.com/Leandro0210/EquipeGo

[R3] Repositorio LIA-221-2026, rama concurrency-parallelism, carpetas 01–07 y first_project. https://github.com/cpjosejesus/lia-221-2026/tree/concurrency-parallelism

[R4] EquipeGo. Evidencias de análisis dinámico DP.11 y auditoría final DP.12. https://github.com/Leandro0210/EquipeGo/tree/master/evidencias

[R5] EquipeGo. Experimentos DP.10 de memoización concurrente. https://github.com/Leandro0210/EquipeGo/tree/master/evidencias/dp10

[R6] OpenStreetMap contributors. Información de licencia y atribución cartográfica. https://www.openstreetmap.org/copyright

[R7] Documentación de la biblioteca estándar de C++: std::thread, std::mutex, std::condition_variable, std::future y std::scoped_lock. https://en.cppreference.com/w/cpp/thread

Anexo A. Parámetros del escenario predeterminado

Categoría	Propiedad	Valor registrado
Grafo	nodes / streets / oneWay	30 / 42 / 7
Restaurantes	restaurants	6
Flota	couriers / bagCapacity / speedKmh	8 / 3 / 30
Flota	startNode	n8
Pedidos	meanIntervalMs / burstMax / maxPending	20.000 / 12 / 50
Pedidos	seed	42
Despacho	quoteTimeoutMs / acceptTimeoutMs	200 ms reales / 600.000 ms simulados
Incidentes	breakdownProbability	0,02 (2 %)
Simulación	durationS / timeScale	3.600 s simulados / 60×

Restaurante	Nodo	pickupSlots	Preparación configurada (ms)
Equipe Pollo	n3	2	60.000–180.000
Brigida Cafe	n28	1	90.000–240.000
Wistupiku	n27	2	45.000–150.000
San Martino	n8	3	60.000–200.000
Fino Juve	n29	2	120.000–240.000
Pollos Ojeda	n12	2	60.000–180.000

Anexo B. Mapa de conceptos de la asignatura (01–07)

Carp.	Concepto	Lugar	Justificación
01	Hilos creados explícitamente y join	main.cpp; Fleet.cpp; Dispatcher.cpp	Sin join existirían hilos activos durante destrucción de objetos.
01	Paralelismo según hardware	Dispatcher.cpp, hardware_concurrency	Sin límite se crearían demasiados workers.

Carp.	Concepto	Lugar	Justificación
02	Valor, referencia y movimiento	std::ref en main; capturas ETA; std::move en OrderBook	Evita copias innecesarias y referencias a temporales.
02	Propiedad única del pedido	Order.hpp y operaciones de movimiento	Impide duplicación accidental de una instancia de pedido.
03	Métodos miembro como destino de thread	Fleet.cpp: &Courier::run	Da un hilo independiente a cada moto.
03	shared_ptr y unique_ptr	Fleet.hpp; RestaurantManager.hpp	Evita destruir monitores mientras los usan repartidores.
04	Trabajo con resultado future	QuoteExecutor::submit/workerLoop	Permite varias cotizaciones con pool reutilizable.
04	Canal promise/future de un uso	SharedDistanceCache.hpp	Comparte un único cálculo de ruta.
04	Deadline y error entre hilos	Dispatcher.cpp::quoteCandidates	Evita bloqueo indefinido por ETA lento o defectuoso.
05	Justificación escrita de no-carreras	README 3.3; protectores de cada módulo	Protección coordinada de datos compartidos.
05	Tipos con copia/movimiento seguro	Order.hpp; PickupGate::Permit	No duplica propiedad ni libera un cupo dos veces.
06	Bloqueos de ámbito, secciones cortas	OrderBook; Courier; QuoteExecutor	Mutex se libera por RAII y el cálculo se hace fuera del lock.
06	Dos mutex sin interbloqueo	Courier::transferAllTo, scoped_lock	Permite mover bolsas entre dos motos sin bloqueo circular.
07	Objetos monitor	OrderBook; Courier; PickupGate	Estado compartido encapsulado bajo mutex.
07	Esperas predicadas y cierre	ordersCv, workCv, CapacitySignal	Evita spinning y permite despertar al detener.

Anexo C. Ruta rápida de reproducción de evidencias

```
# Compilación normal
cmake -S . -B build
cmake --build build -j4
ctest --test-dir build --output-on-failure

# Instrumentación TSAN (en directorio independiente)
cmake -S . -B build-tsan -DENABLE_TSAN=ON \
  -DEQUIPEGO_BUILD_BENCHMARK=ON
cmake --build build-tsan -j4
ctest --test-dir build-tsan --output-on-failure

# Prueba auxiliar 64 motos: usar el JSON de carga documentado
./build-bench/delivery_sim_bench \
  evidencias/dp12/adversarial/valid/large_64/config.json \
  --gui --scale 10 --duration 600 --log /tmp/equipego_64.jsonl
```

Las instrucciones de pruebas y los parámetros exactos por ejecución se conservan en los metadatos de la auditoría del repositorio. Se recomienda registrar el commit, compilador, configuración, semilla, número de repeticiones y resultado de cada nueva corrida.